



GUIA COMPLETO

Agente de IA

para WhatsApp

n8n + Evolution API + Groq (Llama 3)

Automação inteligente de atendimento via WhatsApp
100% gratuito e self-hosted

Stack	n8n + Evolution API + Groq API (Llama 3.3)
Custo	GRATUITO (self-hosted)
Servidor	VPS 2vCPU / 4GB RAM (~R\$60/mês)
Dificuldade	Intermediário — Sem código (no-code)
Tempo de setup	2 a 4 horas

SUMÁRIO

1. Visão Geral — O que vamos construir
2. Arquitetura do Sistema
3. Pré-requisitos
4. Passo 1: Servidor VPS
5. Passo 2: Instalar o n8n (Docker)
6. Passo 3: Instalar Evolution API (WhatsApp)
7. Passo 4: Conectar WhatsApp (QR Code)
8. Passo 5: Configurar Groq API (IA Gratuita)
9. Passo 6: Criar o Fluxo no n8n
10. Passo 7: Configurar o Webhook
11. Passo 8: Prompt do Agente de IA
12. Passo 9: Testar e Validar
13. Passo 10: Domínio e SSL (Produção)
14. Dicas Avançadas e Boas Práticas
15. Tabela de Preços — Quanto Cobrar
16. Troubleshooting — Problemas Comuns

1. VISÃO GERAL — O QUE VAMOS CONSTRUIR

Este guia ensina a criar um **agente de IA completo para WhatsApp** que funciona como atendente virtual 24/7. O agente recebe mensagens, entende a intenção do cliente usando IA (Llama 3.3 via Groq), consulta dados, responde automaticamente e pode executar ações — tudo sem escrever código.

O que o agente faz:

- 1. **Recebe mensagens** de qualquer pessoa no WhatsApp
- 2. **Entende a intenção** usando IA (pergunta, compra, suporte, saudação)
- 3. **Responde automaticamente** com base no contexto
- 4. **Consulta banco de dados** (pedidos, agendamentos, produtos)
- 5. **Encaminha para humano** quando não souber responder
- 6. **Envia mensagens proativas** (lembretes, promoções)

Casos de uso:

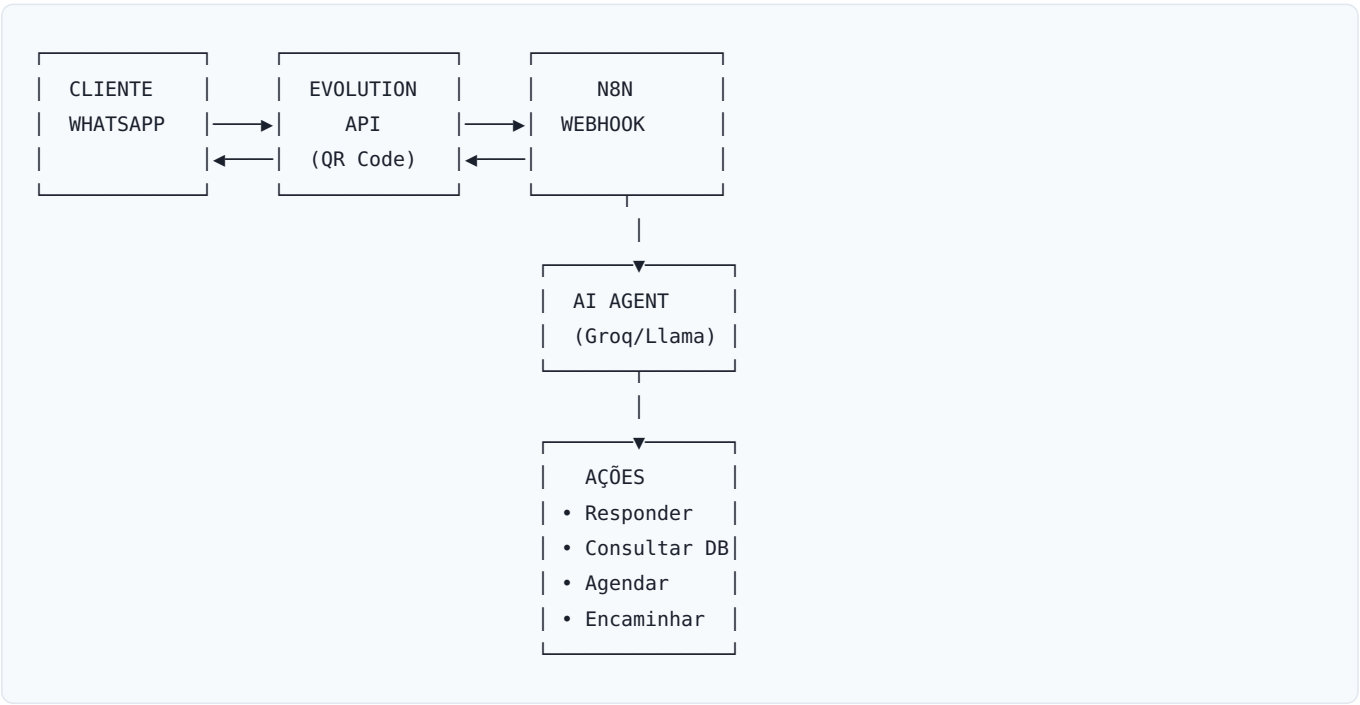
Negócio	Uso do Agente
Clínica médica	Agendar/consultar/cancelar consultas
Restaurante	Cardápio, pedidos, horários
E-commerce	Status de pedido, trocas, dúvidas
Advocacia	Triagem de clientes, agendamento
Imobiliária	Imóveis disponíveis, visitas
Salão de beleza	Agendamento, serviços, preços

 **Por que esta stack?**

- **n8n** — Open-source, visual, sem código. 400+ integrações.
- **Evolution API** — Open-source, conecta ao WhatsApp via QR Code. Sem depender da API oficial cara da Meta.
- **Groq** — API de IA gratuita com Llama 3.3 70B. Muito rápida.

Custo total: R\$ 0 (só o servidor ~R\$ 60/mês)

2. ARQUITETURA DO SISTEMA



Fluxo completo:

1. Cliente envia mensagem no WhatsApp
2. Evolution API recebe e envia via Webhook para o n8n
3. n8n processa a mensagem e envia para o AI Agent
4. IA (Groq/Llama) analisa e decide a resposta
5. n8n executa a ação (responder, consultar DB, agendar)
6. Mensagem de volta via Evolution API → WhatsApp do cliente

Serviço	Porta	Função
Evolution API	8080	Conecta WhatsApp Web, envia/recebe mensagens
n8n	5678	Motor de automação, processa fluxos
PostgreSQL	5432	Banco de dados da Evolution API
Redis	6379	Cache e sessões

3. PRÉ-REQUISITOS

3.1 — Servidor VPS

Especificação		Mínimo	Recomendado
CPU		2 vCPU	4 vCPU
RAM		4 GB	8 GB
Disco		40 GB SSD	80 GB SSD
OS		Ubuntu 22.04 LTS	Ubuntu 24.04 LTS
Custo		~R\$ 60/mês	~R\$ 120/mês

Provedores recomendados: DigitalOcean, Vultr, Hetzner, Contabo, Locaweb

3.2 — Contas gratuitas necessárias

Serviço	Finalidade	Como criar
Groq Cloud	API de IA (Llama 3.3)	console.groq.com — Sign up grátis
n8n (self-host)	Motor de automação	Instalação via Docker (abaixo)
Domínio (opcional)	SSL + acesso externo	Registro br (~R\$ 40/ano) ou Freenom grátis

3.3 — Número de WhatsApp

⚠ **Importante:** Use um número de WhatsApp dedicado (chip separado). Se usar seu número pessoal, ele será "consumido" pelo bot. Recomenda-se um chip pré-pago exclusivo.

4. PASSO 1: CONFIGURAR O SERVIDOR VPS

4.1 — Conectar ao servidor via SSH

```
ssh root@IP_D0_SERVIDOR
```

4.2 — Atualizar o sistema

```
apt update && apt upgrade -y
```

4.3 — Instalar Docker e Docker Compose

```
# Instalar Docker
curl -fsSL https://get.docker.com | sh

# Habilitar Docker no boot
systemctl enable docker
systemctl start docker

# Instalar Docker Compose
apt install docker-compose-plugin -y

# Verificar
docker --version
docker compose version
```

4.4 — Configurar Firewall (UFW)

```
ufw allow 22/tcp      # SSH
ufw allow 80/tcp      # HTTP
ufw allow 443/tcp     # HTTPS
ufw allow 5678/tcp    # n8n
ufw allow 8080/tcp    # Evolution API
ufw enable
```

5. PASSO 2: INSTALAR O N8N

5.1 — Criar diretório e docker-compose.yml

```
mkdir -p /opt/n8n && cd /opt/n8n

cat > docker-compose.yml << 'EOF'
version: '3.8'
services:
  n8n:
    image: n8nio/n8n:latest
    container_name: n8n
    restart: always
    ports:
      - "5678:5678"
    environment:
      - N8N_HOST=n8n.seudominio.com
      - N8N_PORT=5678
      - N8N_PROTOCOL=https
      - WEBHOOK_URL=https://n8n.seudominio.com/
      - GENERIC_TIMEZONE=America/Sao_Paulo
      - TZ=America/Sao_Paulo
      - N8N_SECURE_COOKIE=false
    volumes:
      - n8n_data:/home/node/.n8n
volumes:
  n8n_data:
EOF
```

5.2 — Subir o n8n

```
docker compose up -d

# Verificar se está rodando
docker logs n8n --tail 20
```

5.3 — Acessar a interface

Acesse: **http://IP_DO_SERVIDOR:5678**
Crie a conta de administrador na primeira vez.

6. PASSO 3: INSTALAR EVOLUTION API

6.1 — Criar diretório e docker-compose.yml

```
mkdir -p /opt/evolution && cd /opt/evolution
```

6.2 — Criar o docker-compose.yml completo

```
cat > docker-compose.yml << 'EOF'
version: '3.8'
services:
  postgres:
    image: postgres:15
    container_name: evolution-db
    restart: always
    environment:
      POSTGRES_USER: evolution
      POSTGRES_PASSWORD: evolution2025
      POSTGRES_DB: evolution
    volumes:
      - pg_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  redis:
    image: redis:7-alpine
    container_name: evolution-redis
    restart: always
    command: redis-server --appendonly yes
    volumes:
      - redis_data:/data

  evolution-api:
    image: atendai/evolution-api:latest
    container_name: evolution-api
    restart: always
    depends_on:
      - postgres
      - redis
    ports:
      - "8080:8080"
    environment:
      - SERVER_TYPE=http
      - SERVER_PORT=8080
      - CORS_ORIGIN=*
      - CORS_METHODS=POST,GET,PUT,DELETE
      - CORS_CREDENTIALS=true
      - DEL_INSTANCE=false
      - DATABASE_ENABLED=true
      - DATABASE_PROVIDER=postgresql
      - DATABASE_CONNECTION_URI=postgresql://evolution:evolution2025@postgres:5432/evolution
      - REDIS_ENABLED=true
      - REDIS_URI=redis://redis:6379
      - AUTHENTICATION_TYPE=apikey
      - AUTHENTICATION_API_KEY=sua-chave-secreta-aqui-mude-isso
      - WEBHOOK_GLOBAL_ENABLED=false
      - WEBHOOK_EVENTS_MESSAGES_UPSERT=true
      - WEBHOOK_EVENTS_MESSAGES_UPDATE=true
    volumes:
      - evolution_data:/evolution/store

volumes:
  pg_data:
  redis_data:
  evolution_data:
EOF
```

🔑 **IMPORTANTE:** Altere **AUTHENTICATION_API_KEY** para uma chave segura sua!
Exemplo: minha-chave-super-secreta-2026
Guarde essa chave — você vai precisar dela para conectar ao n8n.

6.3 — Subir a Evolution API

```
docker compose up -d  
  
# Verificar logs  
docker logs evolution-api --tail 30
```

Aguarde até ver: **"Server is running on port 8080"** nos logs.

7. PASSO 4: CONECTAR O WHATSAPP (QR CODE)

7.1 — Acessar o Manager da Evolution API

```
http://IP_DO_SERVIDOR:8080/manager
```

7.2 — Criar uma instância

1. Abra o **Postman** ou use o **curl** diretamente
2. Crie a instância com o seguinte comando:

```
curl -X POST http://IP_DO_SERVIDOR:8080/instance/create \
-H "Content-Type: application/json" \
-H "apikey: sua-chave-secreta-aqui-mude-isso" \
-d '{
  "instanceName": "meu-agente",
  "token": "",
  "number": "",
  "qrcode": true,
  "integration": "WHATSAPP-BAILEYS",
  "reject_call": true,
  "msg_call": "",
  "groups_ignore": true,
  "always_online": true,
  "read_messages": true,
  "read_status": true
}'
```

7.3 — Ler o QR Code

```
# Gerar/obter o QR Code
curl -X GET "http://IP:8080/instance/connect/meu-agente" \
-H "apikey: SUA_CHAVE"
```

A resposta conterá uma imagem em base64. Você pode:

- Salvar como imagem e escanear direto
- Usar o [Postman](#) com a interface visual
- Acessar no browser: **http://IP:8080/manager**

7.4 — Escanear com o WhatsApp

1. Abra o **WhatsApp** no telefone com o chip dedicado
2. Vá em **Configurações** → **Aparelhos conectados**
3. Toque em **Conectar um aparelho**
4. Escaneie o QR Code gerado pela Evolution API
5. Aguarde a conexão (pode levar 30-60 segundos)

7.5 — Verificar conexão

```
curl -X GET "http://IP:8080/instance/connectionState/meu-agente" \
-H "apikey: SUA_CHAVE"
```

- ✓ **Sucesso:** A resposta deve conter "state": "open"
- 🔄 **Conectando:** "state": "connecting" — aguarde e tente novamente
- ✗ **Falha:** Gere um novo QR Code repetindo o passo 7.3

8. PASSO 5: CONFIGURAR GROQ API (IA GRATUITA)

8.1 — Criar conta no Groq

1. Acesse: **console.groq.com**
2. Clique em **"Sign Up"** — pode logar com GitHub ou email
3. Confirme o email

8.2 — Gerar API Key

1. No painel do Groq, vá em **"API Keys"** (menu lateral)
2. Clique em **"Create API Key"**
3. Dê um nome: **"n8n-agente-whatsapp"**
4. Copie a key (começa com gsk_...)

8.3 — Escolher o modelo

Modelo	Velocidade	Qualidade	Gratuito
llama-3.3-70b-versatile	⚡⚡⚡ Muito rápida	★★★★ Excelente	✔ Sim
llama-3.1-8b-instant	⚡⚡⚡⚡ Ultra rápida	★★★ Boa	✔ Sim
mixtral-8x7b-32768	⚡⚡ Rápida	★★★★ Excelente	✔ Sim
gemma2-9b-it	⚡⚡⚡ Rápida	★★★ Boa	✔ Sim

Recomendação: Use **llama-3.3-70b-versatile** — melhor custo/benefício. Muito rápido e inteligente para atendimento.

8.4 — Configurar credencial no n8n

1. No n8n, vá em **"Settings"** (engrenagem) → **"Credentials"**
2. Clique em **"Add Credential"**
3. Pesquise por **"Groq"** ou **"OpenAI"**
4. Cole a API key do Groq
5. Se usar "OpenAI", ajuste a base URL para: **https://api.groq.com/openai/v1**
6. Salve com nome: **"Groq - Agente WhatsApp"**

9. PASSO 6: CRIAR O FLUXO NO N8N

9.1 — Novo workflow

1. No n8n, clique em **"Create Workflow"**
2. Nomeie: **"Agente WhatsApp - Atendimento"**

9.2 — Nó 1: Webhook (receber mensagens)

Adicione um nó **"Webhook"** como trigger:

Parâmetro	Valor
Method	POST
Path	webhook/whatsapp-agente
Authentication	Header Auth → Nome: apikey, Value: sua-chave-evolution

URL do Webhook: **https://n8n.seudominio.com/webhook/whatsapp-agente**
Guarde essa URL — vai configurar na Evolution API depois.

9.3 — Nó 2: Extrair campos da mensagem

Adicione um nó **"Edit Fields (Set)"**:

```
# Campos para extrair:
telefone = {{ $json.data.key.remoteJid.replace('@s.whatsapp.net', '') }}
nome = {{ $json.data.pushName || 'Cliente' }}
mensagem = {{ $json.data.message.conversation || $json.data.message.extendedTextMessage?.text || '' }}
instance = {{ $json.instance }}
tipo = {{ $json.event }}

# Expressão completa no n8n:
telefone:={{ $json.data?.key?.remoteJid?.replace('@s.whatsapp.net', '') || '' }}
mensagem:={{ $json.data?.message?.conversation || $json.data?.message?.extendedTextMessage?.text || '' }}
nome:={{ $json.data?.pushName || 'Cliente' }}
```

9.4 — Nó 3: AI Agent

Adicione um nó **"AI Agent"**:

Parâmetro	Valor
Model	Groq — llama-3.3-70b-versatile
Temperature	0.4 (criativo mas consistente)
System Message	Veja Seção 11 (Prompt)
Prompt (User)	={{ \$json.mensagem }}

9.5 — Nó 4: Enviar resposta

Adicione um nó **"HTTP Request"** para enviar a resposta de volta:

Parâmetro	Valor
Method	POST
URL	http://SEU_IP:8080/message/text/meu-agente

Authentication	Header: apikey = SUA_CHAVE_EVOLUTION
Body (JSON)	Veja abaixo

```
{
  "number": "={{ $('Edit Fields').json.telefone }}",
  "text": "={{ $json.output || $json.text || 'Desculpe, não entendi. Pode repetir?' }}",
  "delay": 1000,
  "linkPreview": false
}
```

10. PASSO 7: CONECTAR EVOLUTION → N8N

10.1 — Configurar webhook no n8n

No mesmo nó Webhook (Passo 9.2), copie a URL de Produção.

10.2 — Registrar webhook na Evolution API

```
curl -X PUT http://IP_DO_SERVIDOR:8080/webhook/set/meu-agente \
-H "Content-Type: application/json" \
-H "apikey: SUA_CHAVE_EVOLUTION" \
-d '{
  "url": "https://n8n.seudominio.com/webhook/whatsapp-agente",
  "webhook_by_server": true,
  "webhook_base64": false,
  "events": {
    "messages.upsert": true,
    "messages.update": false,
    "connection.update": true
  }
}'
```

10.3 — Verificar se o webhook está ativo

```
curl -X GET http://IP_DO_SERVIDOR:8080/webhook/find/meu-agente \
-H "apikey: SUA_CHAVE_EVOLUTION"
```

A resposta deve mostrar a URL configurada.

11. PASSO 8: PROMPT DO AGENTE DE IA

Este é o **System Message** que define o comportamento do agente. Ajuste conforme seu negócio:

```
Você é o atendente virtual da empresa [NOME DA EMPRESA].
Seu nome é [NOME DO AGENTE] e você atende clientes pelo WhatsApp de forma
simpática, profissional e eficiente.

=== REGRAS ===
• Responda SEMPRE em português brasileiro
• Seja breve (máximo 3-4 linhas por mensagem)
• Use emojis com moderação para ser amigável
• NUNCA forneça informações falsas ou invente dados
• Se não souber responda: "Vou encaminhar para um atendente humano"
• Não peça dados sensíveis (senhas, cartões) pelo WhatsApp

=== INFORMAÇÕES DA EMPRESA ===
Nome: [NOME]
Endereço: [ENDEREÇO]
Horário: [HORÁRIO DE FUNCIONAMENTO]
Telefone: [TELFONE]
Serviços: [LISTA DE SERVIÇOS/PRODUTOS]
Preços: [PREÇOS OU "Solicite um orçamento"]

=== FLUXO DE ATENDIMENTO ===
1. SAUDAÇÃO: Cumprimente e pergunte como pode ajudar
2. ENTENDIMENTO: Identifique o que o cliente quer
3. AÇÃO: Responda, agende, consulte ou encaminhe
4. CONFIRMAÇÃO: Confirme se resolveu a dúvida
5. DESPEDIDA: Agradeça e se disponha para mais ajuda

=== CASOS ESPECIAIS ===
• Se for reclamação: "Entendo sua frustração. Vou encaminhar imediatamente."
• Se pedir preço: Informe ou peça para visitar o site/catalogo
• Se quiser agendar: Peça nome, data preferencial e serviço desejado
• Se for spam/indesejado: Ignore educadamente

MENSAGEM DO CLIENTE: {{ $json.mensagem }}
NOME DO CLIENTE: {{ $json.nome }}
```

12. PASSO 9: TESTAR E VALIDAR

9.1 — Testar o webhook manualmente

```
curl -X POST https://n8n.seudominio.com/webhook/whatsapp-agente \
-H "Content-Type: application/json" \
-H "apikey: SUA_CHAVE_EVOLUTION" \
-d '{
  "event": "messages.upsert",
  "instance": "meu-agente",
  "data": {
    "key": {
      "remoteJid": "556299999999@s.whatsapp.net"
    },
    "pushName": "Teste Usuário",
    "message": {
      "conversation": "Olá, gostaria de saber seus horários de funcionamento"
    }
  }
}'
```

9.2 — Enviar mensagem de teste pelo WhatsApp

1. No chip DEDICADO conectado à Evolution, envie uma mensagem para outro número
2. Ou peça para um amigo enviar mensagem para o número do bot
3. Veja no n8n: Workflow → Executions para ver o resultado

9.3 — Checklist de validação

- ☐ Webhook recebe mensagens do WhatsApp
- ☐ n8n processa e salva na execução
- ☐ IA (Groq) responde corretamente
- ☐ Mensagem de volta chega no WhatsApp do cliente
- ☐ Fluxo reconecta ao reiniciar o servidor
- ☐ Rate limiting funciona (evita spam)
- ☐ Fallback para humano ativa quando necessário

13. PASSO 10: DOMÍNIO E SSL (PRODUÇÃO)

10.1 — Apontar domínio para o servidor

No painel do registrador de domínio (Registro.br, Namecheap, etc):

```
Tipo: A
Nome: n8n
Valor: IP_DO_SERVIDOR
TTL: 300
```

```
Tipo: A
Nome: evolution
Valor: IP_DO_SERVIDOR
TTL: 300
```

10.2 — Instalar Nginx + SSL

```
apt install nginx certbot python3-certbot-nginx -y

# Configurar Nginx para n8n
cat > /etc/nginx/sites-available/n8n << 'EOF'
server {
    listen 80;
    server_name n8n.seudominio.com;
    location / {
        proxy_pass http://localhost:5678;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
EOF

# Ativar
ln -s /etc/nginx/sites-available/n8n /etc/nginx/sites-enabled/
nginx -t && systemctl reload nginx

# Obter SSL gratuito
certbot --nginx -d n8n.seudominio.com -m seu@email.com --agree-tos -n

# Repetir para evolution.seudominio.com se necessário
```

14. DICAS AVANÇADAS E BOAS PRÁTICAS

1. **Human-in-the-loop:** Configure um nó "Switch" que mensagens complexas para atendimento humano via Telegram ou email.
2. **Memória de contexto:** Use PostgreSQL no n8n para guardar histórico de conversa e manter contexto entre mensagens.
3. **Rate limiting:** Adicione um nó "IF" limite mensagens por minuto por número para evitar spam.
4. **Horário somente expediente:** Adicione nó que após o expediente envie: "Estamos fora do horário. Retornaremos amanhã!"
5. **Mensagens em massa:** Para disparos marketing, use a Evolution API diretamente com delays entre envios.
6. **Monitoramento:** Configure alertas no n8n (email/Telegram) quando workflow falhar > 3 vezes.
7. **Grupos:** A Evolution API ignora grupos por default (groups=_ignore: true). Pode mudar se quiser moderar grupos.
8. **Mídia:** O suporta envio/recebimento de imagens, áudio, documentos e localização.
9. **Backup:** Faça backup do volume n8n_data regularmente:

```
docker run --rm -v n8n_data:/data -v $(pwd):/backup tar czf /backup/n8n-backup.tar.gz /data
```
10. **Atualização:**

```
docker compose pull && docker compose up -d
```

 para atualizar Evolution API e n8n.

15. TABELA DE PREÇOS – QUANTO COBRAR

Valores praticados no mercado brasileiro para implementação de agentes de IA para WhatsApp em 2025/2026:

Serviço	Mínimo	Médio	Premium
Setup básico (bot responde FAQ)	R\$ 1.000	R\$ 2.000	R\$ 4.000
Agente completo (IA + ações + DB)	R\$ 2.000	R\$ 4.000	R\$ 8.000
+ Integração com sistemas externos	+ R\$ 1.000	+ R\$ 2.000	+ R\$ 4.000
+ Agendamento automático	+ R\$ 500	+ R\$ 1.000	+ R\$ 2.000
+ Envio de mídia (imagens, áudio)	+ R\$ 300	+ R\$ 500	+ R\$ 1.000
+ Dashboard de métricas	+ R\$ 500	+ R\$ 1.000	+ R\$ 2.000
+ Treinamento do modelo (fine-tuning)	+ R\$ 1.000	+ R\$ 2.000	+ R\$ 5.000
Mensalidade (suporte + ajustes)	R\$ 200/mês	R\$ 500/mês	R\$ 1.000/mês
TOTAL (implementação)	R\$ 3.800	R\$ 8.500	R\$ 18.000

💰 **Modelo de negócio:** Cobre implantação (R\$ 2.000-8.000) + mensalidade (R\$ 200-500/mês).
Com 10 clientes = **R\$ 2.000-5.000/mês de receita recorrente.**

16. TROUBLESHOOTING — PROBLEMAS COMUNS

Problema	Causa	Solução
QR Code não aparece	Instância não criada ou já conectada	DELETE a instância e crie novamente
"state": "close"	QR expirou ou conexão caiu	Gere novo QR Code
Webhook não dispara	URL errada ou CORS	Verifique URL no PUT /webhook/set
IA não responde	API key Groq inválida ou rate limit	Verifique key no console.groq.com
Mensagem duplicada	Webhook disparando 2x	Filtre by event= "messages.upsert" AND fromMe=false
Consumo alto de RAM	Evolution API pesada	Adicione ou limite : "DATABASE_PROVIDER=sqlite"
Container reiniciando	Falta de memória na VPS	Adicione swap: fallocate -l 2G /swapfile && mkswap /swapfile && swapon /swapfile
"INVALID_KEY" na Evolution	API key errada	Mude AUTHENTICATION_API_KEY e recrie os containers



PARABÉNS!

Se você seguiu todos os passos, seu agente de IA para WhatsApp está funcionando!

Stack: n8n + Evolution API + Groq (Llama 3.3) = 100% gratuito

Custo operacional: apenas o servidor VPS (~R\$ 60-120/mês)

Valor cobrado ao cliente: R\$ 2.000 a R\$ 18.000 + mensalidade

Documento gerado em Maio/2026 | Corumbá Sistemas